

UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ

FACULTAD DE INGENIERÍA DE SISTEMAS



Framework JS (React), Client-Side Rendering (CSR), JSX, Props & Children, Estrategias de Estilado

Práctica Semana 5

Docente:

Aguilar Coronación, Miguel Elías

Alumno:

Janampa Jaime, Khaled Alejandro

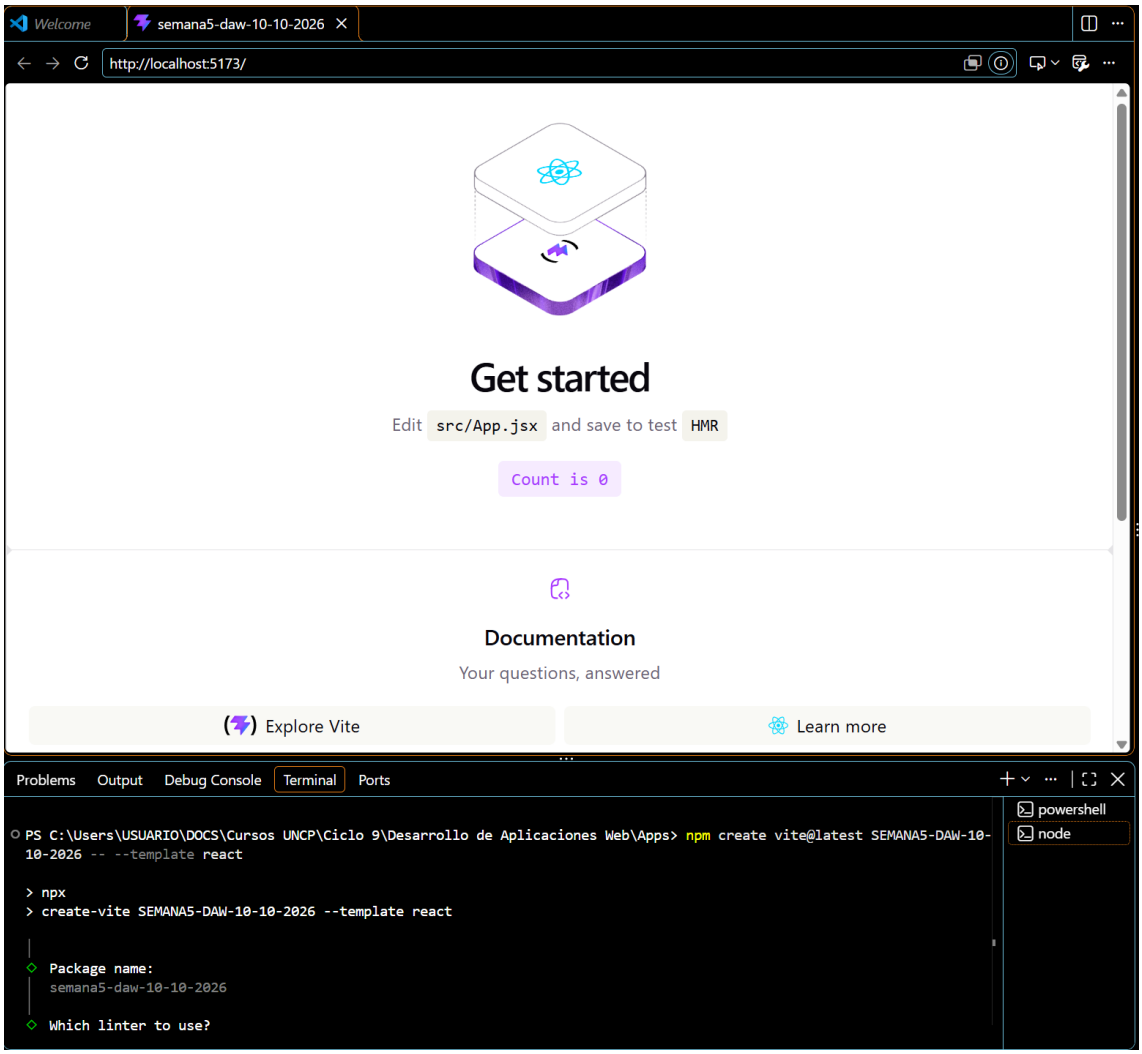
Huancayo – Perú, 2026

OBJETIVO DE LA PRÁCTICA

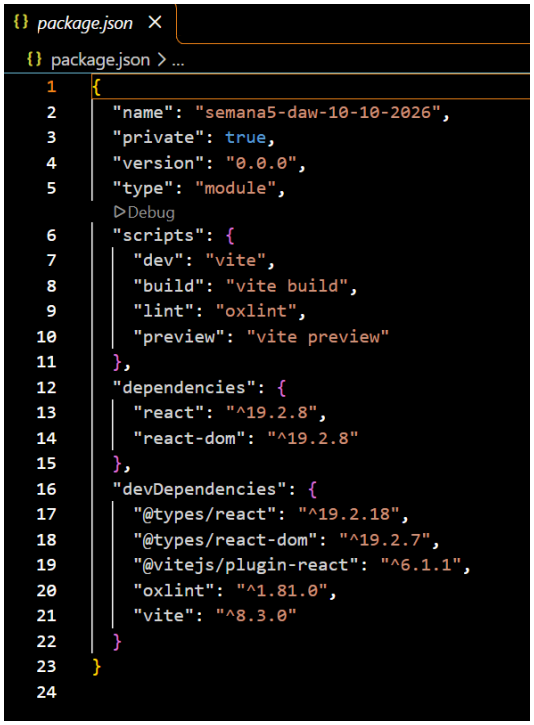
Crear una aplicación frontend de página única (SPA) utilizando un framework JS (React), configurar el entorno con herramientas modernas (Vite + npm/pnpm), implementar arquitectura basada en componentes con JSX, transmitir datos mediante props y composición con children, aplicar estrategias de estilado (CSS Modules / Tailwind / inline) y validar el comportamiento de Client-Side Rendering (CSR) frente al ciclo de vida del DOM.

PASO 1

Scaffold del proyecto: npm create vite@latest, seleccionar React + JavaScript, instalar dependencias, ejecutar npm run dev



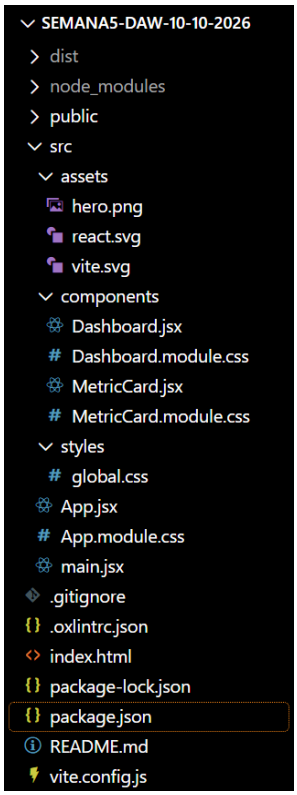
Package.json



```
{
  "name": "semana5-daw-10-10-2026",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "oxlint",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.2.8",
    "react-dom": "^19.2.8"
  },
  "devDependencies": {
    "@types/react": "^19.2.18",
    "@types/react-dom": "^19.2.7",
    "@vitejs/plugin-react": "^6.1.1",
    "oxlint": "^1.81.0",
    "vite": "^8.3.0"
  }
}
```

PASO 2

Limpiar boilerplate, crear src/components/, src/styles/, configurar alias o rutas relativas.
Estructurar App.jsx como orquestador



App.jsx

App.jsx

```jsx
function App() {
  // PASO 2: Limpieza de boilerplate y configuración del componente raíz <App>
  // como único orquestador: aquí se definen y distribuyen los datos de ejemplo.
  return (
    <div className={styles.appShell}>
      <aside className={styles.sidebar} aria-label="Navegación principal">
        <a className={styles.brand} href="#inicio" aria-label="UNCP, inicio">
          <span className={styles.brandMark}>U</span>
          <span className={styles.brandText}>
            <strong>UNCP</strong>
            <small>INGENIERÍA DE SISTEMAS</small>
          </span>
        </a>

        <div className={styles.navLabel}>MENÚ PRINCIPAL</div>
        <nav className={styles.navigation}>
          <a className={`${styles.navItem} ${styles.navItemActive}`} href="#inicio" aria-current="page">
            <Icon name="overview" /> <span>Panel general</span>
          </a>
          <a className={styles.navItem} href="#cursos"><Icon name="courses" /> <span>Cursos</span></a>
          <a className={styles.navItem} href="#actividad"><Icon name="students" /> <span>Estudiantes</span></a>
          <a className={styles.navItem} href="#asistencia"><Icon name="reports" /> <span>Rendimiento</span></a>
        </nav>

        <div className={styles.sidebarBottom}>
          <a className={styles.navItem} href="#configuracion"><Icon name="settings" /> <span>Configuración</span></a>
          <div className={styles.sidebarNote}>
            <span className={styles.noteDot} />
            <div><strong>Ciclo académico</strong><span>2026 - II · En curso</span></div>
          </div>
          <div className={styles.profile}>
            <div className={styles.avatar}>JP</div>
```

PASO 3

Implementar patrón props y children: crear y . Pasar datos estructurados y contenido anidado

Dashboard.jsx

```jsx
import styles from './Dashboard.module.css'

/**
 * Layout presentacional del dashboard. Recibe el contenido de la página a través de props y
 * renderiza su contenido a través de children, manteniendo la propiedad de los datos en App.
 * @param {{ eyebrow: string, title: string, description: string, period: string, children: import('react').ReactNode }} props
 */
function Dashboard({ eyebrow, title, description, period, children }) {
  return (
    <div className={styles.dashboard}>
      <header className={styles.pageHeading}>
        <div>
          <div className={styles.eyebrow}>{eyebrow}</div>
          <h1>{title}</h1>
          <p>{description}</p>
        </div>
        <div className={styles.periodBadge}><span />{period}</div>
      </header>
      {children}
    </div>
  )
}

export default Dashboard
```

MetricCard.jsx

Dashboard.jsx M | MetricCard.jsx ●

src > components > MetricCard.jsx > ...

```jsx
import styles from './MetricCard.module.css'

/**
 * Carta KPI académica reutilizable. El icono/contenido se compone a través de children.
 *
 * @param {{ title: string, value: string, detail: string, caption: string, tone?: 'green' | 'gold', children: import('react').R
 */
function MetricCard({ title, value, detail, caption, tone = 'green', children }) {
  const cardClassName = tone === 'gold'
    ? `${styles.metricCard} ${styles.gold}`
    : styles.metricCard

  return (
    <article className={cardClassName}>
      <div className={styles.cardTop}>
        <span className={styles.title}>{title}</span>
        {children}
      </div>
      <div className={styles.value}>{value}</div>
      <div className={styles.cardBottom}>
        <strong>{detail}</strong>
        <span>{caption}</span>
      </div>
    </article>
  )
}

export default MetricCard
```

PASO 4

Aplicar estrategias de estilado: CSS Modules para , utilidades Tailwind (si se configura) o inline para contrastar scope y colisiones

MetricCard.module.css | *Dashboard.module.css* ×

src > components > # Dashboard.module.css > ...

```css
/* PASO 4: CSS Modules aíslan las clases y evitan colisiones de estilos. */
.dashboard {
  width: 100%;
}

.pageHeading {
  display: flex;
  align-items: flex-end;
  justify-content: space-between;
  padding: 31px 0 23px;
}

.eyebrow {
  color: #829087;
  font-size: 9px;
  font-weight: 700;
  letter-spacing: .1em;
}

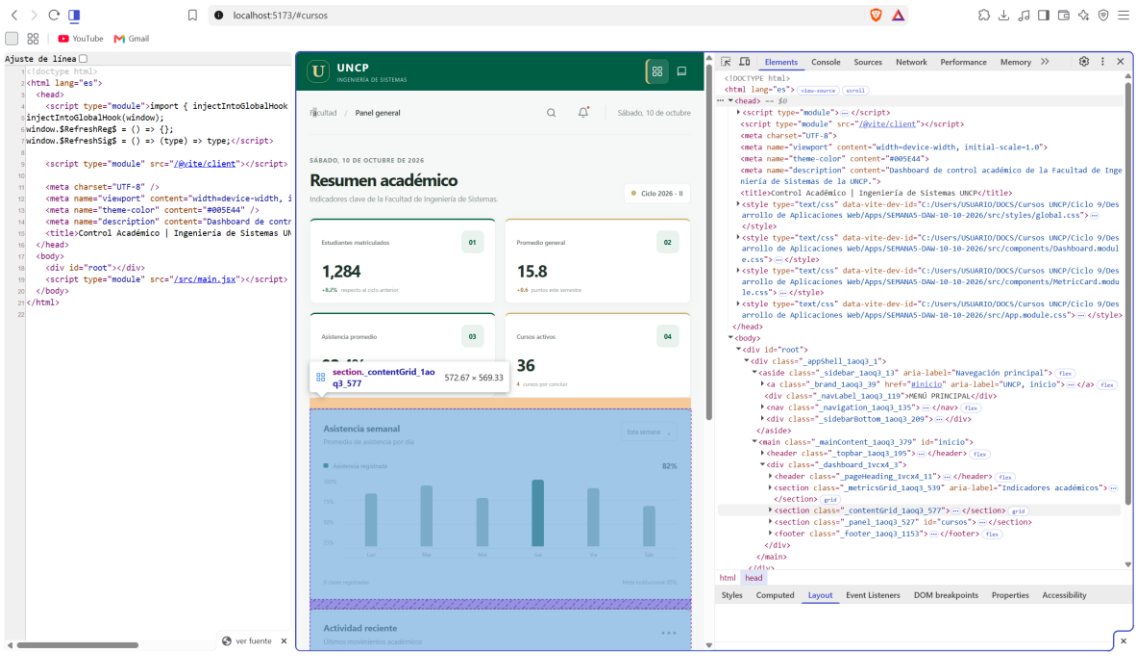
.pageHeading h1 {
  margin: 7px 0 5px;
  color: #1b2e25;
  font-family: inherit;
  font-size: 25px;
  font-weight: 650;
  letter-spacing: -.04em;
}

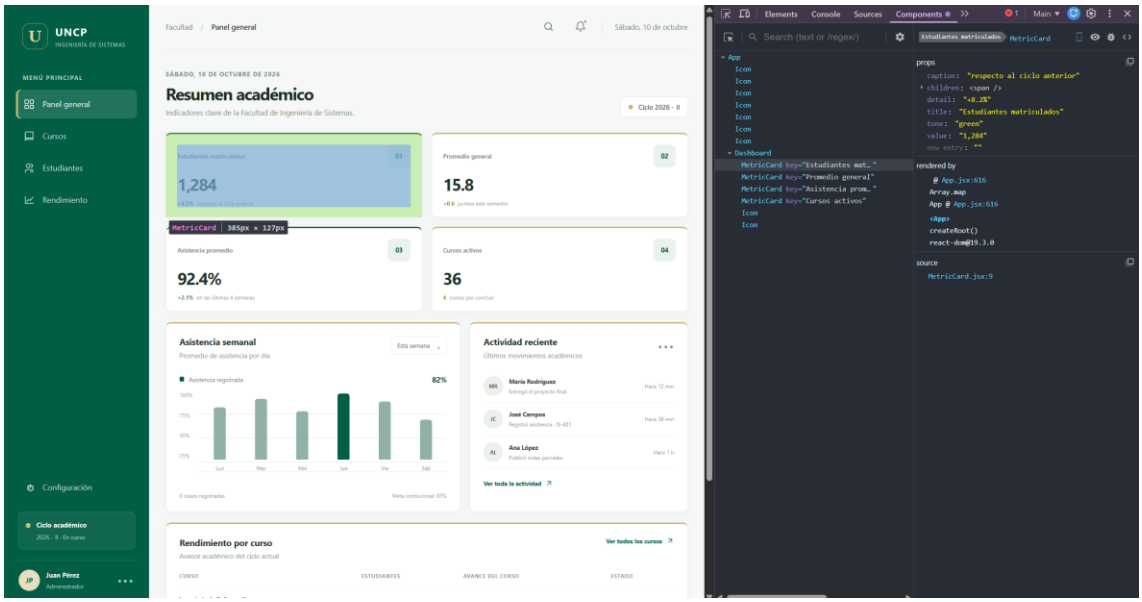
.pageHeading p {
  margin: 0;
  color: #839089;
  font-size: 11px;
}
```

PASO 5

Validar CSR: inspeccionar HTML inicial (View Page Source), verificar hidratación en React DevTools, analizar red (JSchunks) y ciclo de render

En la respuesta HTML inicial solo existe un #root vacío: el contenido del dashboard NO está prerenderizado en el DOM. src/main.jsx ejecuta createRoot(...).render(...) en el navegador y React crea la interfaz en el cliente. Para comparar, inspeccionar "Ver código fuente" frente al DOM del panel Elements después de cargar la página: el segundo sí tendrá el árbol.

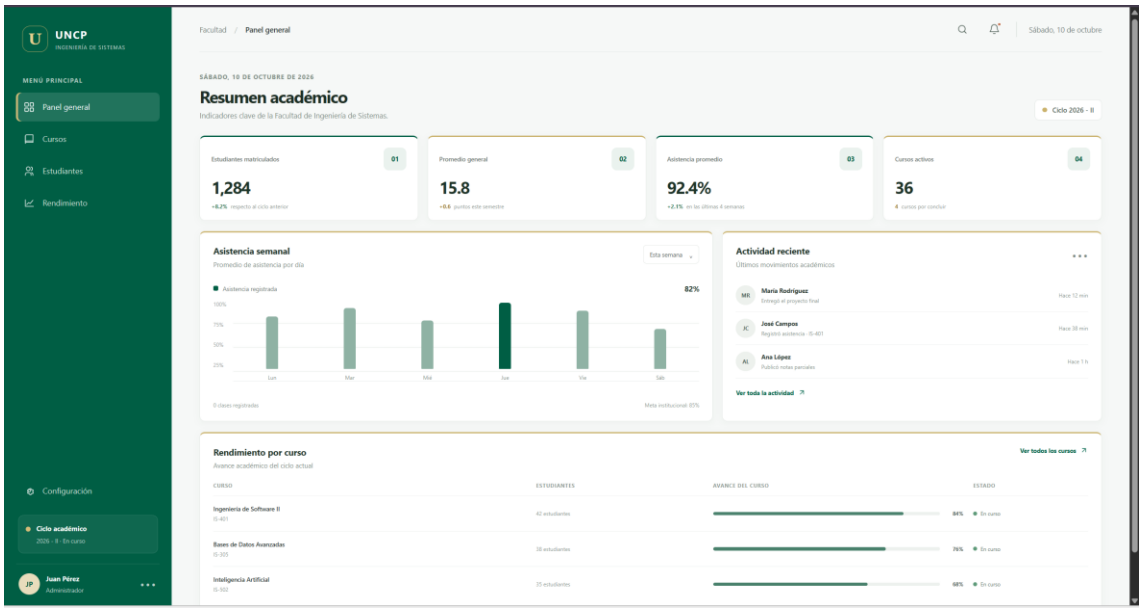




Link: https://github.com/KhaledAle/SEMANA5-DAW-10-10-2026

Anexos

Home


UNCP
INGENIERÍA DE SISTEMAS
MENÚ PRINCIPAL
Panel general
Cursos
Estudiantes
Rendimiento
Configuración
Ciclo académico
2026 - II · En curso
Juan Pérez
Administrador
Facultad / Panel general
Sábado, 10 de octubre
SÁBADO, 10 DE OCTUBRE DE 2026
Resumen académico
Indicadores clave de la Facultad de Ingeniería de Sistemas.
Ciclo 2026 - II
Estudiantes matriculados
01
1,284
+8.2% respecto al ciclo anterior
Promedio general
02
15.8
+0.6 puntos este semestre
Asistencia promedio
03
92.4%
+2.1% en las últimas 4 semanas
Cursos activos
04
36
4 cursos por concluir
Asistencia semanal
Promedio de asistencia por día
Esta semana
Asistencia registrada
82%
100%
75%
50%
25%
Lun
Mar
Mié
Jue
Vie
Sáb
0 clases registradas
Meta institucional: 85%
Actividad reciente
Últimos movimientos académicos
MR
María Rodríguez
Entregó el proyecto final
Hace 12 min
JC
José Campos
Registró asistencia · IS-401
Hace 38 min
AL
Ana López
Publicó notas parciales
Hace 1 h
Ver toda la actividad
Rendimiento por curso
Avance académico del ciclo actual
Ver todos los cursos
CURSO
ESTUDIANTES
AVANCE DEL CURSO
ESTADO
Ingeniería de Software II
IS-401
42 estudiantes
84%
En curso
Bases de Datos Avanzadas
IS-305
38 estudiantes
76%
En curso
Inteligencia Artificial
IS-502
35 estudiantes
68%
En curso